

Material de Estudo — Java: Random e Matrizes

Consolidação da sessão de estudos: classe `Random`, matrizes (arrays 2D) e validação de entrada. Continuação da apostila de fundamentos, seguindo a metodologia DevSuperior.

Sumário

1. Classe `Random`
2. Fórmula de faixa aleatória
3. Matrizes — percurso fundamental
4. Impressão formatada em grade
5. Análise de matriz (maior, menor, soma, média)
6. Isolar uma linha ou uma coluna
7. Validação de entrada
8. Buffer do `Scanner`
9. Dicas e insights
10. Próximos passos

1. Classe `Random`

Gera números pseudoaleatórios. Diferente do `Scanner`, **não lê de lugar nenhum** — produz valores por algoritmo interno, então **nunca tem problema de buffer**.

```
import java.util.Random;

Random r = new Random();

r.nextInt(100)      // 0 a 99   (0 incluído, 100 excluído)
r.nextInt(100) + 1 // 1 a 100  (desloca a faixa para começar em 1)
r.nextInt()         // qualquer int (inclusive negativos gigantes!)
r.nextDouble()      // 0.0 a 1.0 (decimal)
r.nextBoolean()     // true ou false
```

⚠ **Regra do `nextInt(n)`**: vai de `0` até `n-1`, nunca chega em `n` (limite exclusivo — mesma lógica do `i < length` dos loops). Para incluir o 100, seria `nextInt(101)`.

Seed — reprodutibilidade

```
Random r = new Random(42); // seed fixa
```

Com uma seed fixa, o `Random` gera **sempre a mesma sequência**. Essencial para: - **Testes** — resultado repetível a cada execução - **Reprodutibilidade científica** — o mesmo conceito do `random_state=42` em ML/scikit-learn

Sem seed, cada execução é diferente; com seed, é determinística.

2. Fórmula de faixa aleatória

Para gerar um número entre um `min` e um `max` (ambos incluídos):

```
int valor = r.nextInt(max - min + 1) + min;
```

Como a fórmula funciona (construção)

Exemplo: faixa de **50 a 150**.

1. `max - min + 1` → quantos números diferentes existem na faixa. `150 - 50 + 1 = 101` (o `+1` inclui as duas pontas — de 3 a 5 são três números, não dois).
2. `nextInt(101)` → sorteia de 0 a 100 (faixa "deslocada", começa no zero).
3. `+ min` → empurra o resultado para começar no mínimo:
4. sorteou `0` → `0 + 50 = 50` (mínimo)
5. sorteou `100` → `100 + 50 = 150` (máximo)

Analogia: é uma régua de comprimento fixo que você desliza para começar no `min` em vez do zero.

Exemplos

```
// Dado de 6 faces (1 a 6)
r.nextInt(6 - 1 + 1) + 1; // = nextInt(6) + 1

// Nota de 0 a 10
r.nextInt(10 - 0 + 1) + 0; // = nextInt(11)
```

3. Matrizes — percurso fundamental

Matriz em Java é um "**vetor de vetores**" (cada linha é, ela mesma, um vetor 1D).

Declaração

```
int[][] matriz = new int[linhas][colunas]; // dimensões definidas

int[][] matriz = {                                // com valores
    {1, 2, 3},
    {4, 5, 6}
};
```

Percurso com loops aninhados

```
for (int i = 0; i < matriz.length; i++) {          // LINHAS
    for (int j = 0; j < matriz[i].length; j++) {    // COLUNAS da linha i
        matriz[i][j] = ...;
    }
}
```

A distinção-chave

Expressão	O que mede	Analogia
<code>matriz.length</code>	número de linhas	quantos andares tem o prédio
<code>matriz[i].length</code>	número de colunas da linha <code>i</code>	quantos apartamentos no andar <code>i</code>

Por quê? `matriz` é o "vetor de fora" (as linhas). `matriz[i]` é uma linha (um vetor de colunas). Então `.length` na matriz conta linhas; `[i].length` conta as colunas daquela linha.

Regra de ouro

Loop externo (i) controla as linhas. Loop interno (j) controla as colunas. Para cada UMA linha, percorre TODAS as colunas — por isso o j "roda mais rápido".

Índice de acesso

```
matriz[i][j]
//      |  └─ j = coluna (loop interno)
//      └── i = linha  (loop externo)
```

4. Impressão formatada em grade

```
for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        System.out.printf("%6d", matriz[i][j]); // largura 6, alinha colunas
    }
    System.out.println(); // quebra APÓS terminar cada linha
}
```

Dois pontos críticos: - `%6d` reserva 6 caracteres por número → colunas alinhadas mesmo misturando 5 e 250. - `println()` **vazio** vai **depois** do loop interno (dentro do externo) — é o que quebra a linha entre uma linha da matriz e a próxima. Se ficar dentro do interno, cada número cai numa linha; se ficar fora do externo, tudo gruda.

Alternativa rápida (debug):

```
System.out.println(Arrays.deepToString(matriz)); // [[1, 2], [3, 4]]
// deepToString para 2D; toString simples imprimiria endereços de memória
```

5. Análise de matriz (maior, menor, soma, média)

```

int soma = 0;
int maior = 0, menor = 0;
int linMaior = -1, colMaior = -1, linMenor = -1, colMenor = -1;

for (int i = 0; i < matriz.length; i++) {
    for (int j = 0; j < matriz[i].length; j++) {
        soma += matriz[i][j];

        if (i == 0 && j == 0) {          // PRIMEIRA célula: inicializa
            maior = matriz[i][j];
            menor = matriz[i][j];
            linMaior = i; colMaior = j;
            linMenor = i; colMenor = j;
        } else {
            if (matriz[i][j] > maior) {    // dois if INDEPENDENTES
                maior = matriz[i][j];
                linMaior = i; colMaior = j;
            }
            if (matriz[i][j] < menor) {
                menor = matriz[i][j];
                linMenor = i; colMenor = j;
            }
        }
    }
}

double media = (double) soma / (matriz.length * matriz[0].length);

```

Pontos críticos

- **Inicialização em `i == 0 && j == 0`** (os DOIS índices no zero — a primeira célula). Em vetor era só `i == 0`; em matriz precisa dos dois, senão reinicializa em toda a primeira linha.
- **Dois `if` independentes** (não `if/else`) — um valor pode não ser nem o maior nem o menor (fica no meio). O `else` só serve para categorias exclusivas (par/ímpar), não para maior/menor.
- **Coordenadas separadas** — cada valor rastreado precisa da sua própria dupla (`linMaior / colMaior` e `linMenor / colMenor`), senão uma sobrescreve a outra.
- **Média:** divisor é `linhas × colunas` (total de células), com `(double)` para não truncar.
- **Cuidado com soma duplicada:** acumule a soma em UM lugar só. Somar de novo no bloco de inicialização conta a célula `[0][0]` duas vezes.

6. Isolar uma linha ou uma coluna

Para UMA linha ou UMA coluna, basta **um loop** (a fatia já é 1D). O aninhamento só é necessário para a matriz inteira.

Regra de ouro

O índice que você FIXA é a dimensão escolhida. O índice que VARIA no loop é a dimensão percorrida.

Uma linha (fixa `i`, varia `j`)

```

int linha = 1;    // FIXA
for (int j = 0; j < matriz[linha].length; j++) {
    soma += matriz[linha][j];
}

```

Uma coluna (fixa `j`, varia `i`)

```
int coluna = 2; // FIXA
for (int i = 0; i < matriz.length; i++) {
    soma += matriz[i][coluna];
}
```

Quero	Fixo	Varia	Limite do loop
Linha	<code>i</code>	<code>j</code>	<code>matriz[i].length</code> (colunas)
Coluna	<code>j</code>	<code>i</code>	<code>matriz.length</code> (linhas)

⚠ **Cuidado copy-paste:** ao duplicar o bloco de linha para fazer coluna, troque TUDO — `linhas ↔ colunas`, `i ↔ j`, e o limite da validação. Esquecer um gera bug silencioso (validar coluna contra `linhas` aceita/rejeita índices errados).

7. Validação de entrada

Nível básico — while com intervalo

```
int escolha = sc.nextInt();
while (escolha < 0 || escolha > limite - 1) {
    System.out.println("Digite entre 0 e " + (limite - 1));
    escolha = sc.nextInt();
}
```

Protege contra índice fora do alcance (evita `ArrayIndexOutOfBoundsException`). Mas **quebra** se o usuário digitar letra (`InputMismatchException`).

Nível robusto — try/catch com parseInt

Lê como texto e converte, tratando qualquer entrada inválida sem crashar:

```
int valor;
while (true) {
    String entrada = sc.nextLine().trim();
    try {
        valor = Integer.parseInt(entrada);
        if (valor >= 0 && valor < limite) break; // dentro do intervalo → sai
        System.out.println("Fora do intervalo. Digite de 0 a " + (limite - 1));
    } catch (NumberFormatException e) {
        System.out.println("Digite um número válido.");
    }
}
```

À prova de letra, símbolo e número fora do intervalo. Bônus: como usa só `nextLine()`, evita o problema de buffer.

Design: String vs int na entrada

- **Menu (categorias como 1/2)** → ler como `String` é mais tolerante: qualquer digitação vira texto e o `while` pede de novo, em vez de crashar com `nextInt()`.
- **Índices (números num intervalo)** → `int` / `parseInt` é natural, já que a intenção é numérica.

8. Buffer do Scanner

Problema **exclusivo do Scanner** (o `Random` não tem). Ocorre ao misturar `nextInt()` / `nextDouble()` com `nextLine()`:

```
int n = sc.nextInt();    // lê o número, deixa o "\n" (Enter) no buffer
sc.nextLine();          // ← limpa o "\n" órfão
String s = sc.nextLine(); // agora lê corretamente
```

Situação	Precisa limpar?
Random gerando números	✗ Nunca
Só nextInt / nextDouble seguidos	✗ Não
nextInt → nextLine	✓ Sim (caso clássico)
Só nextLine seguidos	✗ Não

💡 Ler **tudo** com `nextLine()` + `parseInt()` elimina o problema de buffer de vez, porque nunca mistura os dois estilos.

9. Dicas e insights

1. **Conferir resultado contra o valor real** pega bugs: somar a matriz na mão e comparar com o output revelou uma soma duplicada. Validar em vez de assumir é hábito profissional.
2. **Não espalhe a mesma operação** por lugares diferentes — a soma acumulada em dois pontos contou `[0][0]` duas vezes. Cada responsabilidade em um lugar só.
3. **if/else vs dois if**: use `if/else` para categorias exclusivas e exaustivas (par/ímpar); use `if` independentes quando um item pode satisfazer ambas ou nenhuma (maior/menor).
4. **Fixar índice = escolher dimensão**. Linha → fixa `i`; coluna → fixa `j`. Uma dimensão fixa, um loop.
5. **Ctrl + Alt + 0** (IntelliJ) remove imports não usados; **Ctrl + Alt + L** reformata o código.
6. **Random com seed** = testes reprodutíveis. Mesmo conceito de `random_state` em ML.
7. **Pensar "e se o usuário digitar besteira?"** já é mentalidade acima de júnior. Validar entrada com try/catch protege a experiência.

10. Próximos passos

Tópicos identificados como evolução natural, para o estudo intermediário:

- **Métodos / decomposição**: quebrar `main` longo em funções com responsabilidade única (`greencherMatriz()` , `analisarLinha()` , `validarEntrada()`). Elimina a duplicação entre casos linha/coluna.
- **POO**: modelar com classes (ex.: uma classe `Matriz` com métodos próprios) em vez de tudo procedural no `main`.
- **Collections e Streams**: `Collections.max/min` , `List` , e `stream().average()` para maior/menor/média de forma concisa (`flatMap` para achatar matriz 2D). Requer entender `Integer` vs `int` (coleções só aceitam objetos).
- **Testes automatizados (JUnit)**: validar casos-limite de forma repetível.
- **Diagonal principal** (`i == j`) e operações por todas as linhas/colunas: exercícios de matriz ainda por praticar.

Material em evolução — atualizado conforme o avanço nos estudos.

Jader Silva · [GitHub](#) · [LinkedIn](#)